

Text Representation 2

Week 7 – Programming

Professor: Misuk Kim
Teaching Assistant: Minjoo Son
minjoo77@hanyang.ac.kr



Contents

1. Semantic Search
2. Retrieval Augmented Generation
3. Assignment

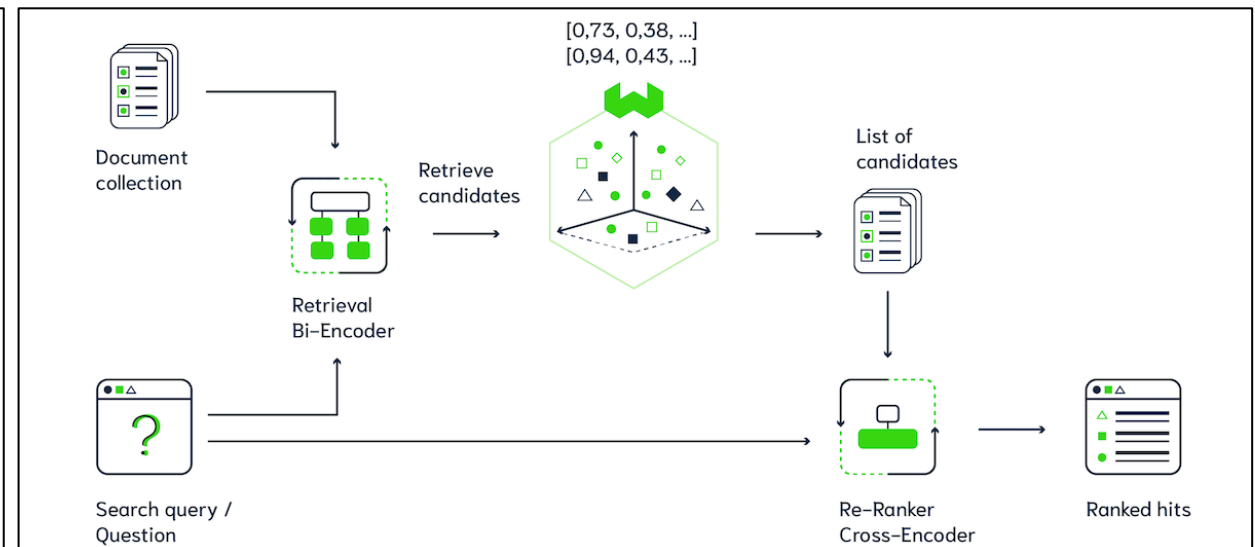
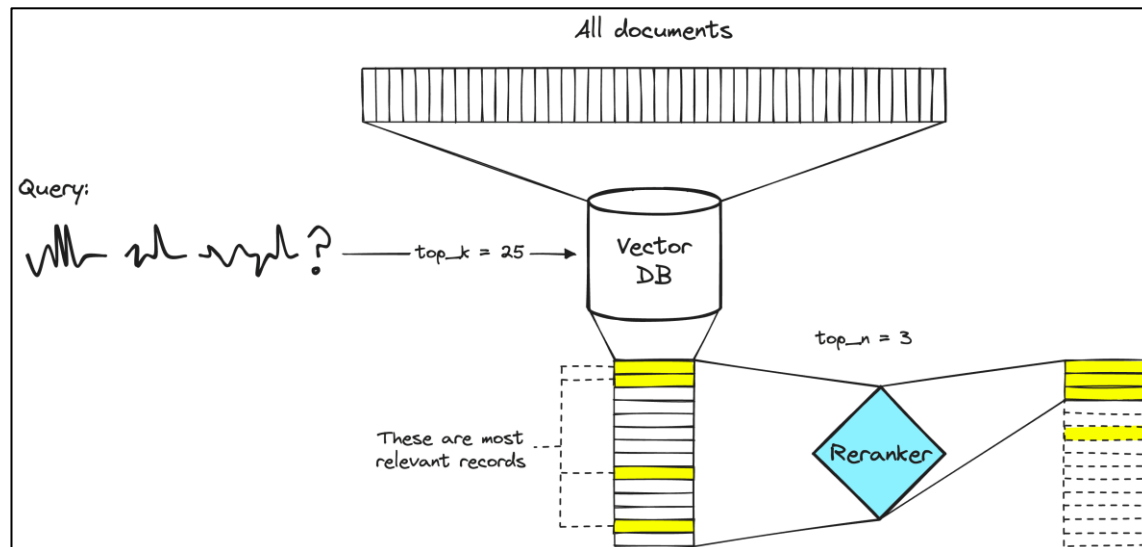
❖ Introduction

- Why This Lab for Text Representation Class?
 - This lab explores **semantic search** – a practical application where **text representation quality directly determines system performance**.
 - While we'll focus on building a real retrieval system, you'll see firsthand how different representation strategies (embeddings, architectures, fine-tuning) impact results.
- Connection to Text Representation
 1. **Dense embeddings in action**: See how 3072-dimensional vectors capture semantic meaning
 2. **Representation comparison**: OpenAI vs open-source models (3072-dim vs 768-dim)
 3. **Architecture matters**: Bi-encoders (independent representations) vs Cross-encoders (joint representations)
 4. **Representation learning**: Fine-tuning embeddings for domain-specific tasks

By the end, you'll understand not just what text representations are, but how they work in real applications and why certain design choices matter.

❖ What is Semantic Search?

- **Semantic search** understands the *meaning* of queries and documents, going beyond keyword matching.
- Example
 - **Query:** "Who analyzed the biopsies?"
 - **Relevant Document:** "Rutgers University biochemists examined the tissue samples..."
 - **Why it matches:** Same meaning, different words



❖ Programming Practice

- https://drive.google.com/file/d/1kAvUUI8u98TPT_kdPwSxbGIHjlvgtgQsg/view?usp=drive_link
(1. semantic search.ipynb)

❖ Putting the R(etrieval) in RAG: Building Production Systems

- Connection to Previous Lab and Text Representation Class
 - In the previous lab, you learned about the **theory and evaluation of semantic search systems**:
 - **Bi-encoders vs Cross-encoders** (different representation architectures)
 - **Embedding quality comparison** (OpenAI 3072-dim vs open-source 768-dim)
 - **Two-stage retrieval pipelines** (fast retrieval + accurate reranking)
 - **Evaluation metrics** (Recall@K, accuracy)

You discovered that representation quality directly impacts downstream task performance—improving representations and architectures.

❖ Putting the R(etrieval) in RAG: Building Production Systems

- What This Lab Adds: Production RAG Systems
 - This lab teaches you to **build production-ready RAG (Retrieval-Augmented Generation) systems**. While the previous lab focused on evaluation with pre-loaded datasets, this lab shows you how to:
 1. **Index real-world data** (web scraping, data cleaning, embedding generation)
 2. **Build scalable vector databases** (Pinecone setup, batch processing)
 3. **Implement end-to-end retrieval pipelines** (from raw text to query results)
 4. **Handle production concerns** (deduplication, metadata management, incremental updates)

❖ Putting the R(etrieval) in RAG: Building Production Systems

- Text Representation in RAG Systems

- RAG systems are where **text representation meets real applications**. Every design choice you make about representations affects system behavior:

Representation Decision	Impact on RAG System
Embedding model (OpenAI vs open-source)	Retrieval quality (+25% in previous lab)
Vector dimensionality (768 vs 1536 vs 3072)	Storage cost vs accuracy tradeoff
Architecture (bi-encoder vs cross-encoder)	Speed vs accuracy tradeoff
Fine-tuning strategy	Domain adaptation quality
Chunking strategy	Granularity of retrieved context

- This lab demonstrates that **representation engineering is system engineering**—your choices about how to represent text cascade through every component of the system.

❖ What is RAG (Retrieval-Augmented Generation)?

- **RAG extends LLMs with external knowledge** by retrieving relevant information before generating responses.

[1] ENCODE DOCUMENTS

- Additional documents are encoded using embedding model
- Convert text documents into vector representations

[2] INDEX

- Store document embeddings in vector database
- Build searchable index for fast retrieval

[3] ENCODE QUERY User Query: "I lost my Medicare card"

- Convert user query to vector embedding using the same embedding model

[4] SIMILARITY SEARCH (RETRIEVAL)

- Search vector database for similar document vectors
- Compare query embedding with indexed document embeddings
- Return top-K most relevant passages
- Retrieved Context: "To replace your Medicare card, visit ssa.gov/myaccount..."

[5] RETRIEVE SIMILAR DOCUMENTS

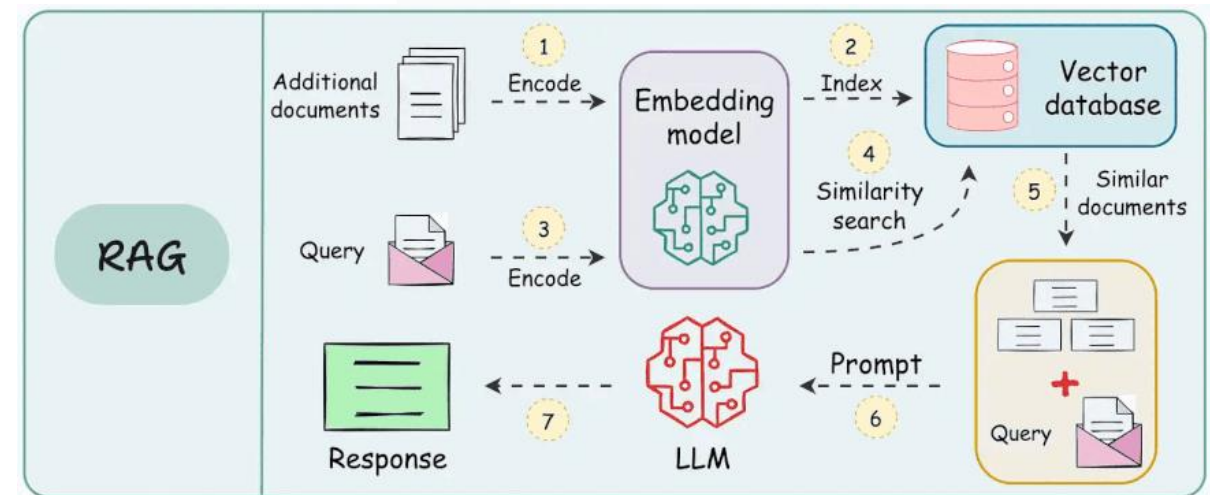
- Extract the most relevant documents from vector database
- Prepare context for augmentation stage

[6] AUGMENTATION (PROMPT CONSTRUCTION)

- Combine retrieved context with original query
- Format prompt: [Retrieved Context] + [User Query]
- Prepare input for LLM

[7] GENERATION

- Send augmented prompt to LLM (GPT-4, Claude, etc.)
- Generate response grounded in retrieved context
- Produce final answer
- Answer: "Based on SSA.gov, you can replace your Medicare card by..."



❖ What is RAG (Retrieval-Augmented Generation)?

- Why This Lab Focuses on Retrieval

- -. If you retrieve irrelevant documents:

- LLM will generate incorrect or hallucinated responses
 - User receives wrong information
 - System loses trust
 - The previous lab showed you **how to evaluate retrieval quality**. This lab shows you **how to build high-quality retrieval systems** that can serve real applications.

❖ Lab Overview

- What We're Building
 - A production-ready semantic search system for Medicare FAQ documents:
 - **Scrape** Medicare FAQs from SSA.gov
 - **Process** and clean text data
 - **Embed** documents using OpenAI's text-embedding models
 - **Index** embeddings in Pinecone vector database
 - **Query** with semantic search
 - **Manage data** (upload, update, delete)
 - Key Text Representation Concepts in This Lab
 1. **Dense Vector Embeddings**: Converting text → fixed-size vectors that capture semantic meaning
 2. **Vector Similarity Search**: Finding similar texts via cosine similarity in embedding space
 3. **Metadata Management**: Storing original text alongside embeddings for retrieval
 4. **Batch Processing**: Efficient embedding generation for large datasets
 5. **Deterministic Hashing**: Using MD5 hashes for deduplication and consistency

❖ Programming Practice

- https://drive.google.com/file/d/1AKc-UDvQNqGiavfVyzC6jFlxrqmINmcS/view?usp=drive_link
(2. retrieval augmented generation.ipynb)

❖ Building Your Own RAG System

- In this assignment, you'll build a complete Retrieval-Augmented Generation (RAG) system from scratch. You'll implement key components, compare different text representation strategies, and evaluate system performance.
 - https://colab.research.google.com/drive/1VK99uT-91cJFAuEyu3FbSGFi_Pnyf8eG?usp=drive_link (3. Assignment.ipynb)
- Submission Instructions:
 - Completed Jupyter Notebook with all code cells executed
 - Written responses to conceptual questions
 - Performance comparison graphs
 - Save your notebook as: 'Assignment_YourName_YourStudentID.ipynb'
 - Submit to: 'Code Practice Assignment 3 – Text Representation 2' section under 'Assignments'
 - **Deadline: Monday, October 21, 11:59 PM**

Q & A

Thank you for your attention. Any questions are welcome!

Minjoo Son

